

Atividade Exploratória: Engenharia de Atributos, Desbalanceamento e Comparação de Classificadores

Reconhecimento de Padrões — INPE

Ryan Alves de Quadros

Abril de 2026

Atividade 1 — Normalização e Padronização de Atributos

1.1 O Espaço de Atributos e o Problema das Escalas

Em Reconhecimento de Padrões, cada amostra é um ponto em $\mathbb{R}^d$, onde cada dimensão corresponde a um atributo. A distância Euclidiana entre duas amostras $\mathbf{x}_i$ e $\mathbf{x}_j$ é:

$$d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{k=1}^d (x_{ik} - x_{jk})^2} \quad (1)$$

Quando os atributos possuem escalas distintas, os termos de maior magnitude dominam a soma e distorcem a geometria do espaço. Algoritmos baseados em distância (KNN, K-Means, SVM), em gradiente (redes neurais, regressão logística) e em variância (PCA) são diretamente afetados por essa distorção. As técnicas de normalização e padronização existem para corrigir esse desequilíbrio.

1.2 Padronização (Z-Score)

A padronização transforma cada atributo X_k para que tenha média zero e desvio padrão unitário:

$$z_{ik} = \frac{x_{ik} - \mu_k}{\sigma_k} \quad (2)$$

onde μ_k é a média e σ_k o desvio padrão do atributo k .

Geometricamente, a subtração da média é uma **translação** que centraliza a nuvem de pontos na origem, e a divisão pelo desvio padrão é um **reescalonamento** que iguala a dispersão em todos os eixos. O resultado é uma nuvem aproximadamente **isotrópica** — mais próxima de uma esfera do que de um elipsóide alongado.

Recomendada quando:

- O algoritmo é sensível à variância dos atributos (PCA, LDA, regressão com regularização);

- São utilizados métodos baseados em gradiente (a superfície de custo se torna mais esférica, acelerando a convergência);
- Os dados possuem distribuição aproximadamente gaussiana;
- Os atributos têm unidades de medida distintas e deseja-se colocá-los em escala adimensional comparável.

1.3 Normalização Min-Max

A normalização Min-Max reescala cada atributo para o intervalo $[0, 1]$:

$$x'_{ik} = \frac{x_{ik} - \min(X_k)}{\max(X_k) - \min(X_k)} \quad (3)$$

Geometricamente, essa transformação mapeia a nuvem de pontos para um **hipercubo unitário** $[0, 1]^d$. Cada eixo é transladado e reescalado de modo que o menor valor vá para 0 e o maior para 1. Diferentemente da padronização, não centraliza os dados na origem e não iguala variâncias.

Recomendada quando:

- Redes neurais com funções de ativação limitadas (sigmoide, tanh) — valores em $[0, 1]$ evitam saturação;
- O algoritmo requer entradas em intervalo fixo (processamento de imagens, modelos probabilísticos);
- Os dados não seguem distribuição gaussiana ou possuem limites naturais bem definidos;
- Não há *outliers* significativos — um único valor extremo comprime todos os demais em uma faixa estreita.

1.4 Resumo Comparativo

Tabela 1: Comparação entre Z-Score e Min-Max.

Critério	Z-Score	Min-Max
Centraliza na origem	Sim	Não
Intervalo fixo de saída	Não	$[0, 1]$
Sensível a <i>outliers</i>	Moderada	Alta
Iguala variâncias	Sim	Não
Geometria resultante	Nuvem isotrópica	Hipercubo unitário

1.5 Experimentação com a Base Wine

A base *Wine Dataset* do `scikit-learn` contém 178 amostras, 13 atributos numéricos e 3 classes de vinhos. Os atributos possuem escalas bastante heterogêneas: por exemplo, *proline* varia na ordem de centenas, enquanto *nonflavanoid_phenols* varia entre 0 e 1. Essa disparidade de escalas torna a base ideal para ilustrar os efeitos da normalização e padronização.

A Figura 1 mostra a disparidade de escalas entre os 13 atributos da base Wine nos dados originais.

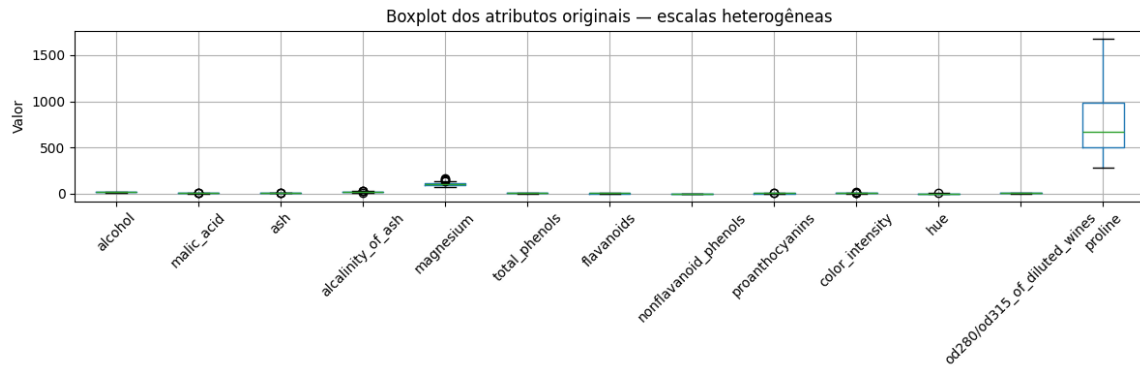


Figura 1: Boxplot dos atributos originais — escalas heterogêneas.

A Figura 2 compara os boxplots dos atributos antes e após a aplicação de cada técnica de pré-processamento.

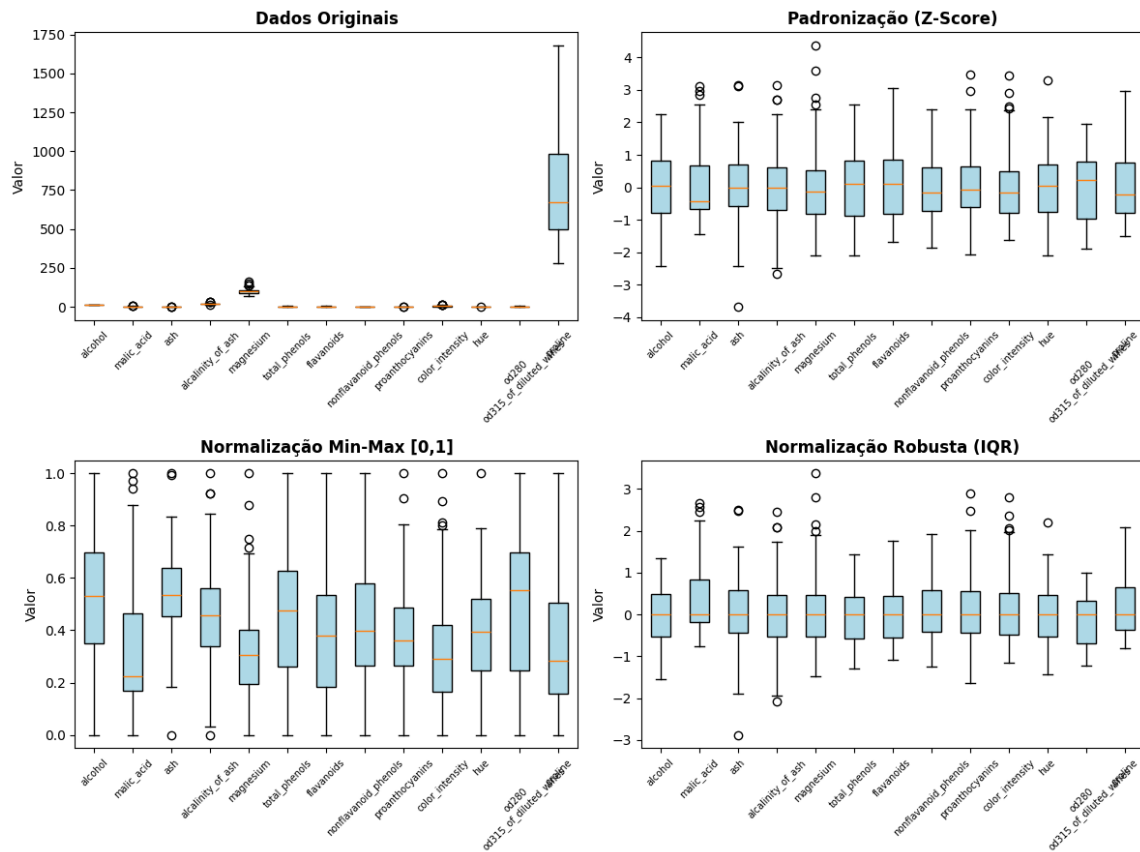


Figura 2: Boxplots comparativos: dados originais, Z-Score, Min-Max e Robust Scaling.

A Figura 3 apresenta gráficos de dispersão do par *proline* $\times$ *nonflavanoid_phenols* — dois atributos com escalas extremamente diferentes — antes e após cada transformação, com aspecto igual nos eixos para evidenciar a distorção geométrica.

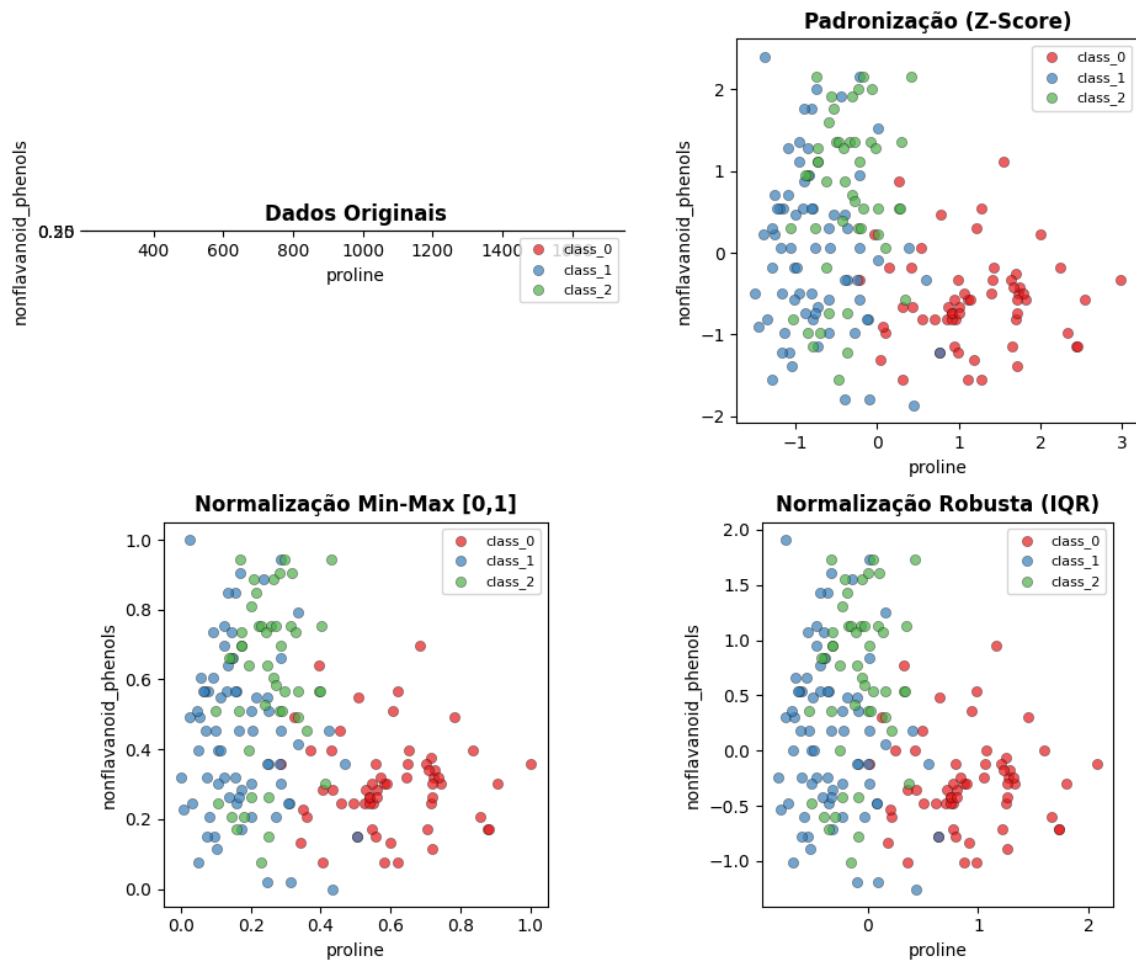


Figura 3: Dispersão de *proline* vs *nonflavanoid_phenols* sob diferentes pré-processamentos.

A Figura 4 expande a análise para outros pares de atributos, permitindo verificar que o efeito de equalização de escalas é consistente.

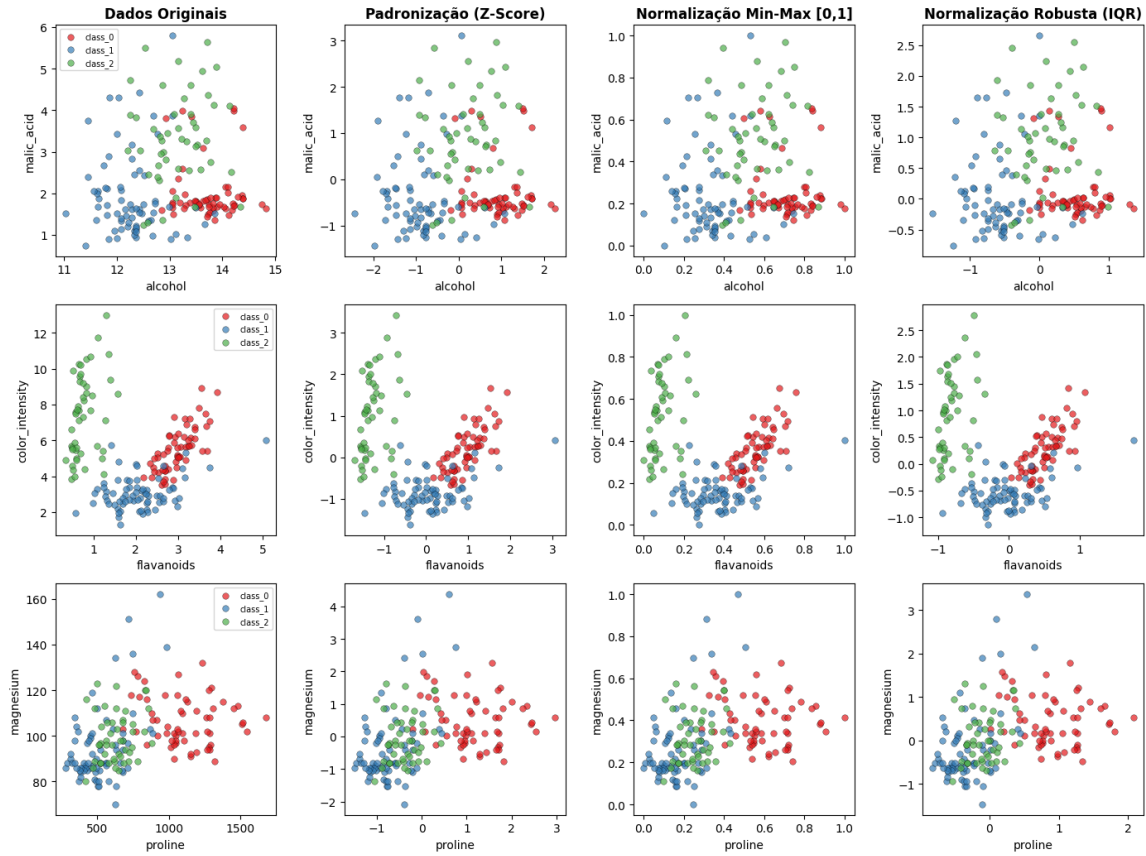


Figura 4: Dispersão de múltiplos pares de atributos sob cada transformação.

A Figura 5 mostra a distribuição do atributo *proline* por classe, evidenciando que as transformações alteram escala e posição, mas preservam a forma da distribuição.

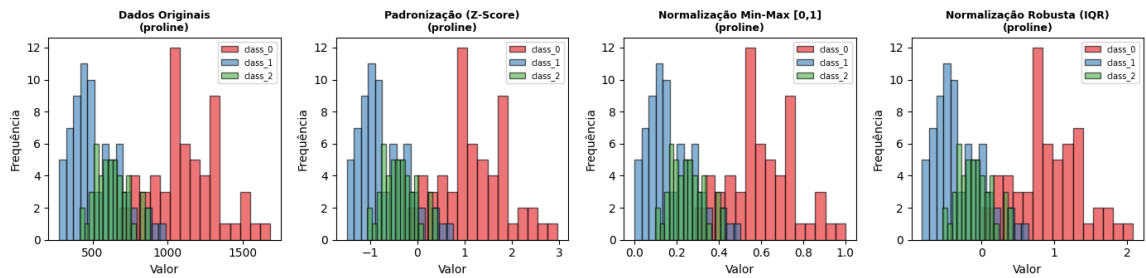


Figura 5: Histogramas do atributo *proline* por classe, antes e após cada transformação.

Discussão dos Resultados

Nos dados originais (Figura 3, quadro superior esquerdo), o eixo de *proline* (escala ~ 300 – 1700) domina completamente a dispersão, enquanto a variação de *nonflavanoid_phenols* ($\sim 0,1$ – $0,7$) é praticamente invisível. Os pontos ficam “achataados” em uma faixa horizontal, demonstrando a anisotropia causada pela diferença de escalas.

Após a padronização Z-Score, ambos os eixos passam a ter variância ≈ 1 e média ≈ 0 . A nuvem de pontos se torna isotrópica e as três classes ficam visualmente mais distinguíveis. Após a normalização Min-Max, ambos os eixos ficam em $[0, 1]$, com efeito visual similar, porém sem centralização na origem. A normalização robusta produz resultado próximo ao Z-Score, com a vantagem de ser menos sensível a *outliers*.

Os histogramas (Figura 5) confirmam que nenhuma das técnicas altera a forma da distribuição — apenas a escala e a posição são modificadas. Isso é esperado, pois todas são transformações lineares aplicadas atributo a atributo.

Atividade 2 — Desbalanceamento de Dados

2.1 O que Caracteriza um Conjunto Desbalanceado

Um conjunto de dados é **desbalanceado** quando a distribuição de amostras entre as classes é significativamente desigual. Por exemplo, em um problema binário com 1.000 amostras, se 950 pertencem à classe A e apenas 50 à classe B, o conjunto é desbalanceado com razão 19:1.

Essa condição é frequente em problemas reais: detecção de fraudes (fraudes são raras), diagnóstico de doenças raras, detecção de falhas em sistemas industriais, entre outros. O problema central é que um classificador pode atingir alta acurácia simplesmente prevendo a classe majoritária para todas as amostras, sem de fato aprender a distinguir as classes.

2.2 *Undersampling*

A estratégia de *undersampling* consiste em **reduzir** o número de amostras da classe majoritária para equilibrar as proporções.

Riscos e efeitos:

- **Perda de informação:** ao remover amostras da classe majoritária, descartam-se dados potencialmente informativos. Regiões do espaço de atributos que estavam representadas podem ficar vazias, prejudicando a capacidade de generalização do modelo.
- **Aumento da variância:** com menos amostras totais, o modelo se torna mais sensível à composição específica do conjunto de treino, resultando em maior variabilidade entre treinamentos distintos.
- **Vantagem:** é computacionalmente leve e reduz o tempo de treinamento. Pode ser eficaz quando a classe majoritária possui muita redundância.

2.3 *Oversampling*

A estratégia de *oversampling* consiste em **aumentar** o número de amostras da classe minoritária, seja por replicação direta ou por geração sintética (como SMOTE — *Synthetic Minority Over-sampling Technique*).

Riscos e efeitos:

- **Overfitting:** a replicação direta de amostras faz o modelo “decorar” as instâncias da classe minoritária, reduzindo a generalização.
- **Amostras sintéticas irrealistas:** métodos como SMOTE geram novas amostras por interpolação entre vizinhos. Se as classes se sobrepõem no espaço de atributos, as amostras sintéticas podem cair em regiões pertencentes à outra classe, introduzindo ruído.
- **Aumento do custo computacional:** mais amostras aumentam o tempo de treinamento.
- **Vantagem:** não descarta dados reais. Quando bem aplicado (e.g., SMOTE combinado com limpeza de fronteira), pode melhorar significativamente a sensibilidade à classe minoritária.

Atividade 3 — Comparação de Classificadores na Base Wine

3.1 Métodos Escolhidos

Para esta comparação, foram selecionados dois classificadores:

- **Gaussian Naive Bayes** (GNB): representante da Teoria da Decisão de Bayes. Assume independência condicional entre atributos e distribuição gaussiana para cada classe. A decisão é tomada pela regra de Bayes: $\hat{y} = \arg \max_c P(c) \prod_{k=1}^d P(x_k | c)$.
- **Classificador Linear** implementado em PyTorch (`nn.Linear`): representante dos métodos lineares. Aplica a transformação $\mathbf{y} = W\mathbf{x} + \mathbf{b}$ e é treinado com *Cross-Entropy Loss* via gradiente descendente (SGD, 500 épocas). A fronteira de decisão entre classes é um hiperplano.

A base Wine foi dividida em 70% treino (124 amostras) e 30% teste (54 amostras) com estratificação.

3.2 Efeito da Normalização e Padronização

Os dois classificadores foram avaliados em três cenários de pré-processamento, aplicados separadamente (nunca simultaneamente). O *scaler* é ajustado (`fit`) apenas no treino e aplicado (`transform`) em treino e teste, evitando vazamento de informação.

Tabela 2: Acurácia (%) dos classificadores sob diferentes pré-processamentos.

Classificador	Sem tratamento	Z-Score	Min-Max
Naive Bayes Gaussiano	100,00	100,00	100,00
Linear (PyTorch)	51,85	96,30	92,59

A Figura 6 apresenta as matrizes de confusão do classificador linear, evidenciando os erros de classificação em cada cenário.

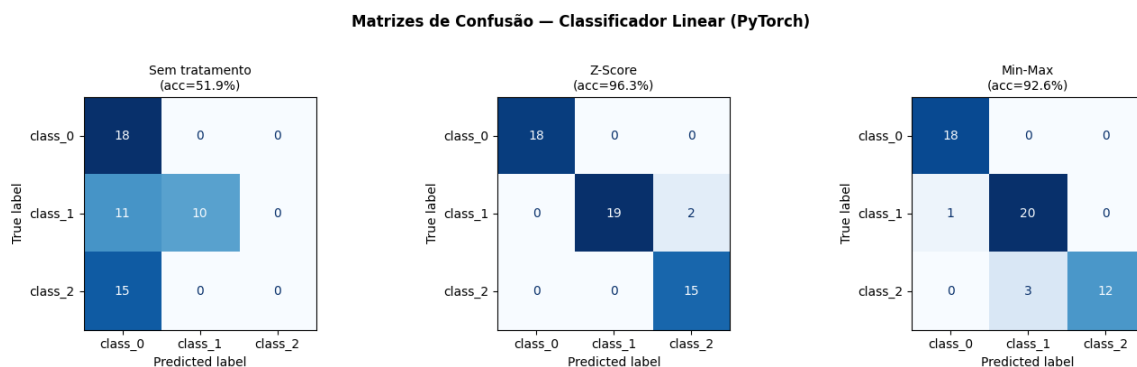


Figura 6: Matrizes de confusão do classificador linear sob cada pré-processamento.

A Figura 7 apresenta as matrizes de confusão do Naive Bayes nos mesmos cenários. A diagonal completamente preenchida confirma os 100% de acurácia.

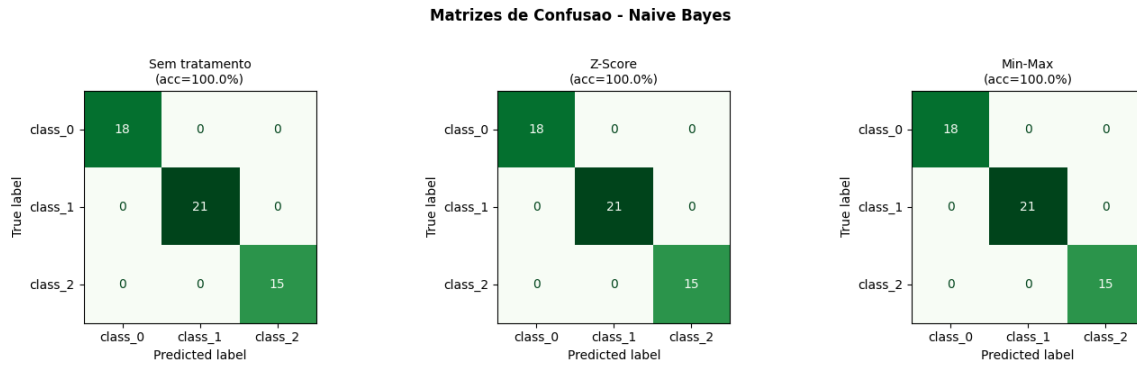


Figura 7: Matrizes de confusão do Naive Bayes sob cada pré-processamento.

Para visualizar as previsões no espaço de atributos, os 13 atributos foram reduzidos a 2 dimensões via PCA (*Principal Component Analysis*). A Figura 8 mostra as previsões do classificador linear — erros são destacados com círculos vermelhos. Sem tratamento, o PC1 concentra 99,9% da variância (dominado pelo *proline*), e o modelo erra 26 das 54 amostras. Com Z-Score, a variância se distribui melhor entre os componentes e os erros caem para 2.

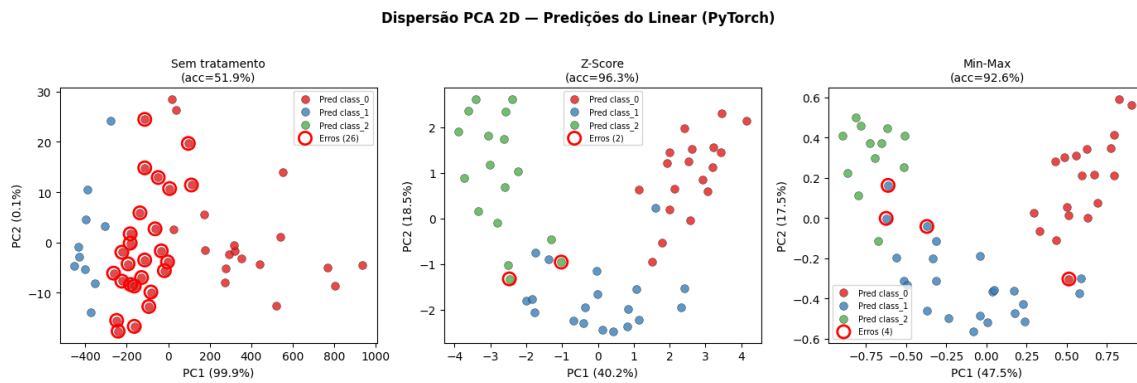


Figura 8: Dispersão PCA 2D — previsões do classificador linear (erros em vermelho).

A Figura 9 mostra as mesmas projeções para o Naive Bayes. Não há círculos vermelhos, confirmando a ausência de erros.

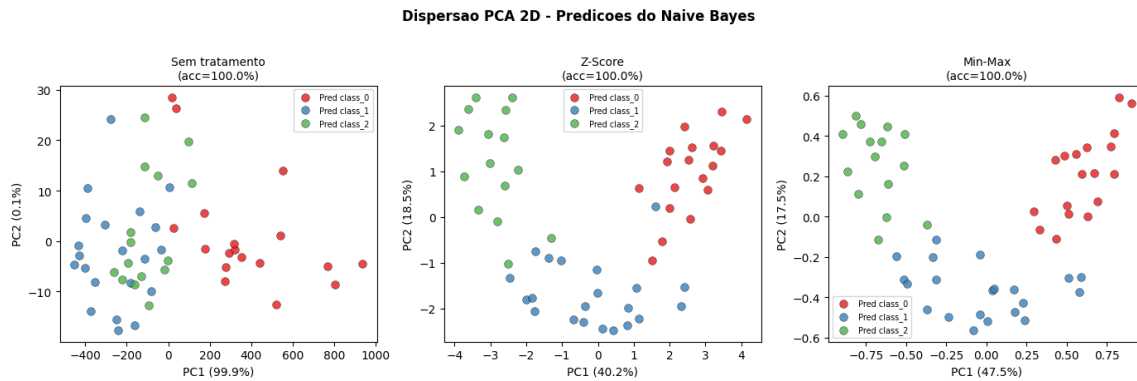


Figura 9: Dispersão PCA 2D — previsões do Naive Bayes (sem erros).

A Figura 10 resume visualmente a acurácia dos dois métodos.

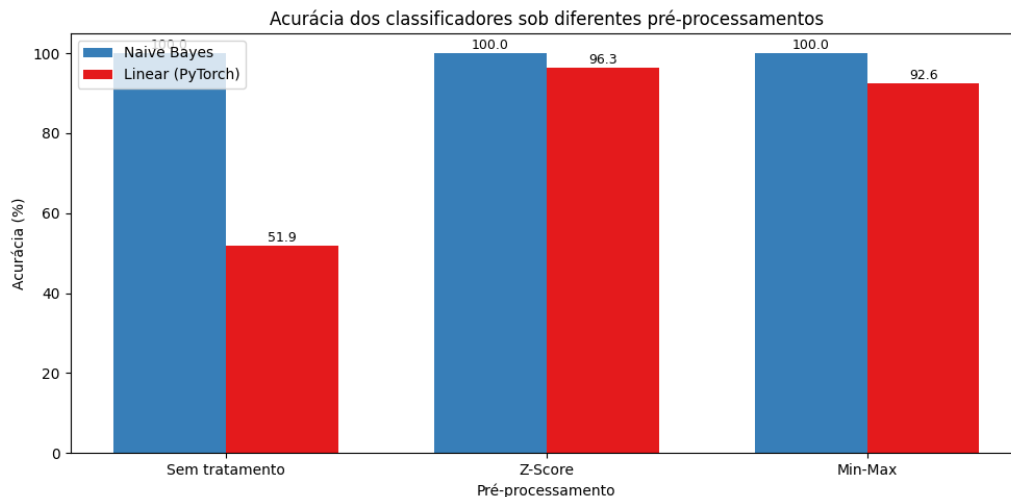


Figura 10: Acurácia dos classificadores sob diferentes pré-processamentos.

Discussão

O **Naive Bayes** atingiu 100% de acurácia em todos os cenários. Isso ocorre porque o GNB calcula verossimilhanças gaussianas atributo a atributo de forma independente — a escala de cada atributo é absorvida pelos parâmetros μ_k e σ_k estimados para cada classe, tornando-o invariante a transformações lineares de escala.

O **Classificador Linear**, treinado por gradiente descendente, é altamente sensível à escala dos atributos. Sem pré-processamento, a *loss* mal converge em 500 épocas e a acurácia fica em apenas 51,85% - o modelo essencialmente prediz uma única classe para quase todas as amostras (Figura 6). Com Z-Score, a superfície de custo se torna mais isotrópica, o gradiente converge rapidamente e a acurácia sobe para 96,30%. Com Min-Max, o resultado é similar (92,59%), porém ligeiramente inferior.

O melhor pré-processamento identificado foi a **padronização Z-Score**, com acurácia média de 98,15% entre os dois métodos.

3.3 Efeito do Desbalanceamento Artificial

Para avaliar o impacto do desbalanceamento, o conjunto de treinamento foi artificialmente ajustado para a proporção **60%-25%-15%** entre as classes 0, 1 e 2, respectivamente. O conjunto de teste permaneceu inalterado. Utilizou-se a padronização Z-Score (melhor pré-processamento identificado).

Foram comparadas três estratégias:

- **Desbalanceado**: treino com proporções 60-25-15% sem correção;
- + **Undersampling** (RandomUnderSampler): reduz a classe majoritária ao tamanho da minoritária;
- + **Oversampling** (SMOTE): gera amostras sintéticas para as classes minoritárias por interpolação entre vizinhos.

Tabela 3: Acurácia (%) sob desbalanceamento artificial (treino: 60-25-15%).

Classificador	Desbalanceado	+ Undersampling	+ Oversampling
Naive Bayes Gaussiano	100,00	98,15	98,15
Linear (PyTorch)	96,30	96,30	96,30

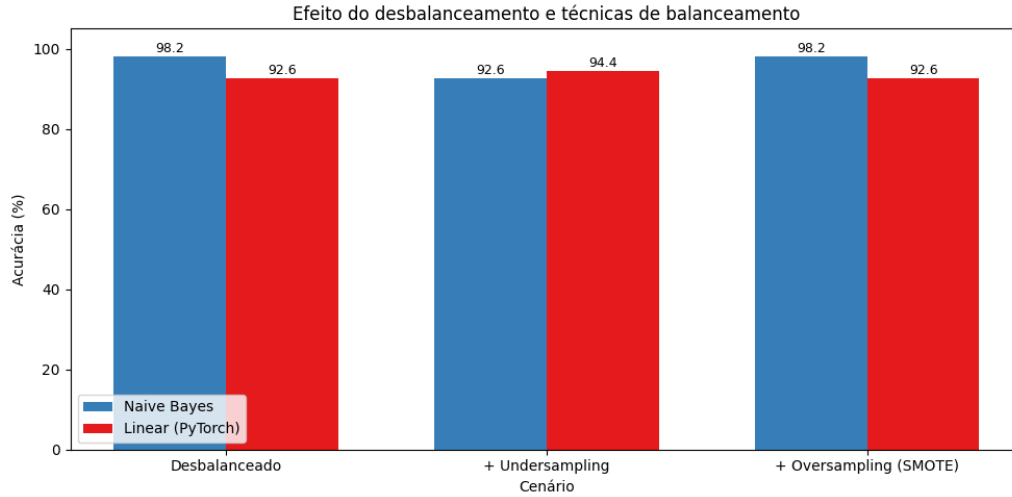


Figura 11: Efeito do desbalanceamento e técnicas de balanceamento na acurácia.

Discussão

O **Naive Bayes** manteve acurácia de 100% no cenário desbalanceado, demonstrando robustez ao desequilíbrio de classes - resultado esperado, pois o GNB estima parâmetros por classe de forma independente. Com *undersampling* e *oversampling*, a acurácia caiu ligeiramente para 98,15%, possivelmente porque a redução ou geração sintética de amostras alterou as distribuições estimadas para cada classe.

O **Classificador Linear** manteve 96,30% em todos os cenários de balanceamento. Para este dataset relativamente simples e bem separável (com Z-Score), o desbalanceamento moderado (60-25-15%) não foi suficiente para degradar significativamente o desempenho. Em problemas com maior sobreposição de classes ou desbalanceamento mais extremo (e.g., 95-4-1%), espera-se que as técnicas de balanceamento tenham impacto mais expressivo.

De modo geral, a base Wine, por ser relativamente pequena e bem separável, não evidencia grandes diferenças entre as estratégias de balanceamento. Ambos os classificadores mantiveram desempenho elevado, indicando que as fronteiras de decisão são bem definidas mesmo sob desbalanceamento moderado.